

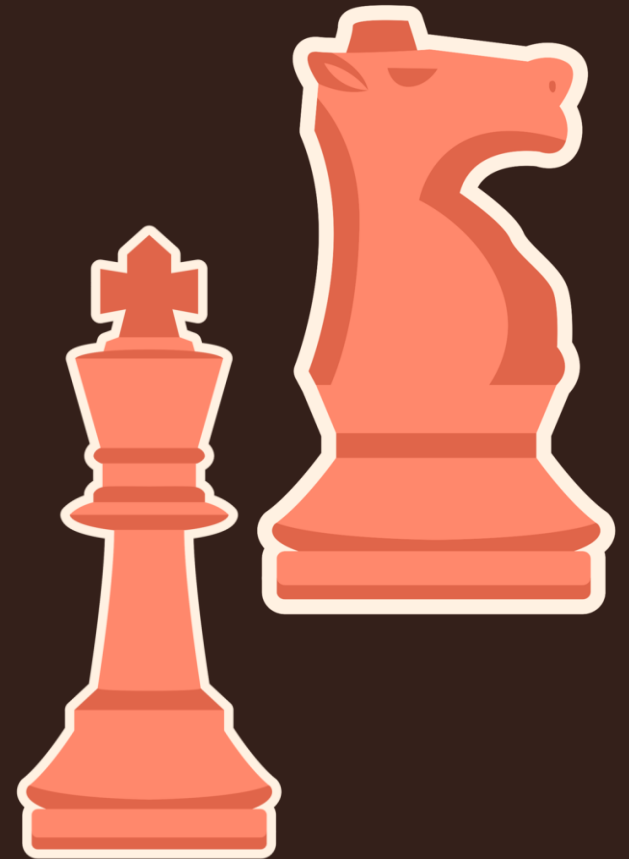


piChess

piChess · pronounced like "peaches"

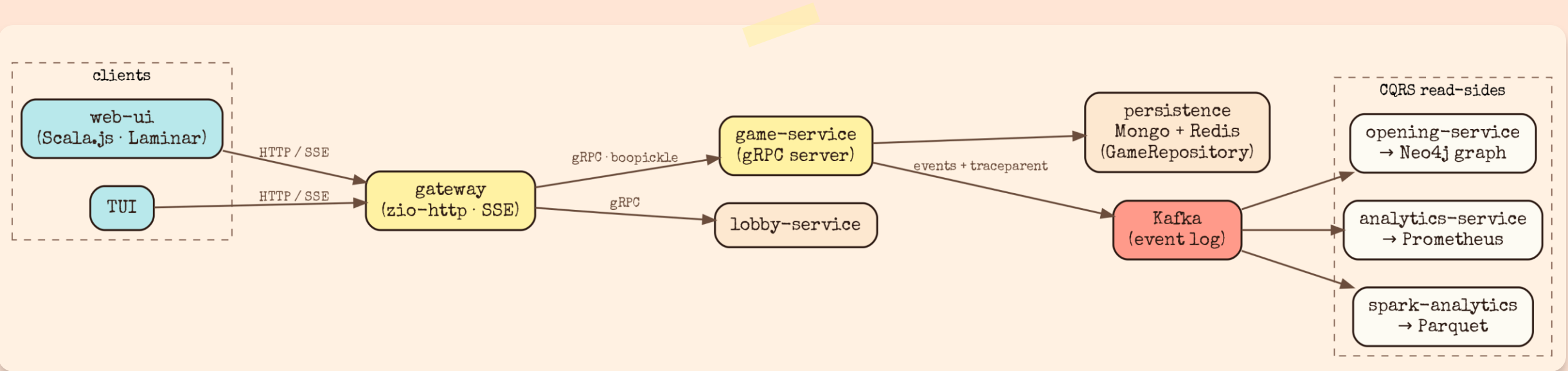
We were asked for a chess game. We did all of that —
then kept going. Here's the "kept going."

Software Architecture · built in Scala 3 + ZIO by Claude et al. · Final Presentation
Julian Nohl · Philippe Westenfelder · Jonas Kromer



ARCHITECTURE

The system at a glance



Six services, one ZIO vocabulary: clients to gateway to game-service to Kafka to CQRS read-sides.

A research project in one phase

The brief said one sentence — "add a computer opponent." We shipped an engine we rate at ~2000 Elo (real-game-conservative) — self-test estimates ~2350 — with a self-trained neural net.

≈2000

Elo (conservative)

self-test ~2350; real game lower

NNUE

self-trained net

PyTorch (768→128)×2→1, int16

HCE+NNUE

hybrid eval

phase-tapered α 0.4 → 0.6

α - β + TT

negamax search

quiescence, NMP, LMR, SEE, ext.

LazySMP

parallel search

+ adaptive time management

3 runtimes

one pure library

vs-CPU, Lichess, NowChess (ADR-020)

Bitboards, magic & threat maps

one Long = 64 squares

0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
1	1	1	1	1	1	1	1
0	0	0	0	0	0	0	0

bit set = a white pawn is here

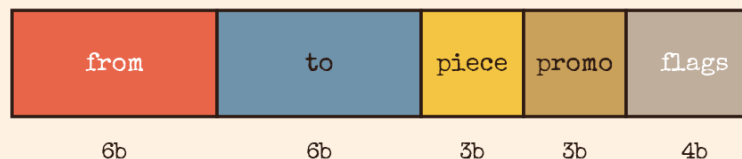
12 of these Longs are the whole board



source of
truth

a whole move packed into one 32-bit Int

no object allocation in the search loop



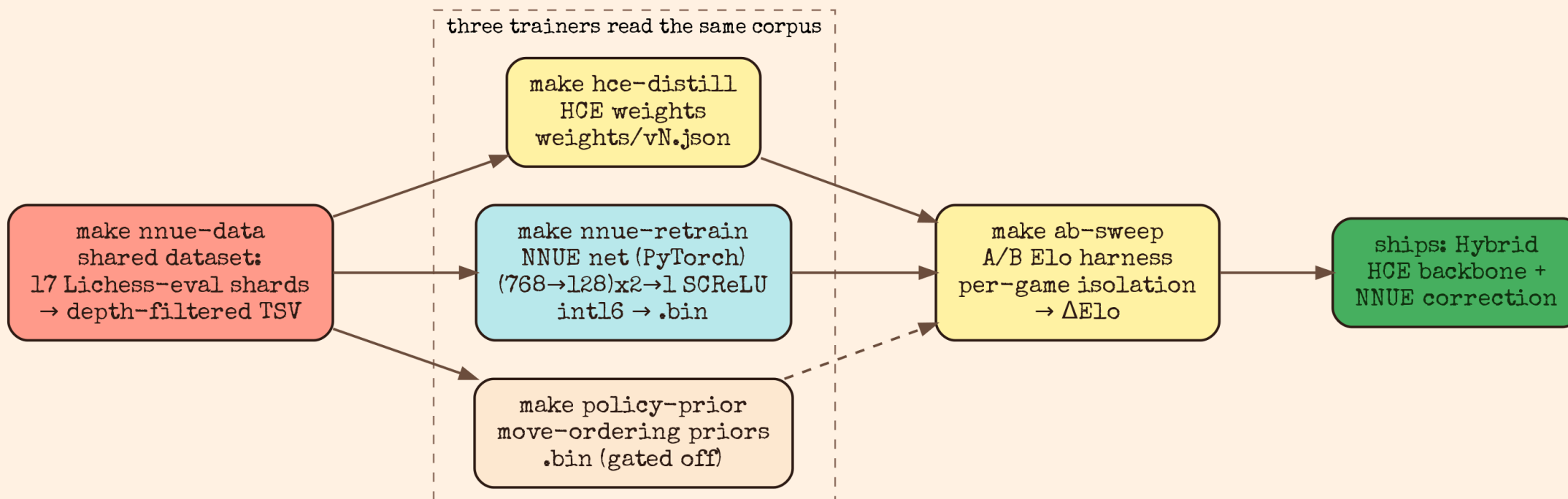
...in code

```
opaque type Bitboard = Long          // 64 squares, one per bit
extension (bb: Bitboard)
  inline def |(o: Bitboard): Bitboard = bb | o
  inline def &(amp;o: Bitboard): Bitboard = bb & o

// sliding attacks: one multiply-shift indexes a magic table
def rookAttacks(sq: Int, occ: Long): Long =
  val idx = (((occ & Mask(sq)) * Magic(sq)) >>> Shift(sq)).toInt
  RookAttackTable(sq)(idx)

// a whole move packed into one 32-bit Int — no allocation in search
inline def encode(from: Int, to: Int, promo: Int): Int =
  from | (to << 6) | (promo << 12)
```

We built the whole forge



One shared Lichess-eval dataset feeds three trainers; ab-sweep A/Bs at the 2 s/move budget and the strongest Hybrid net ships.

ZIO all the way — and it paid off

- **We chose the ZIO equivalents**

- Lecture said Akka HTTP / Slick / Akka Streams — we used zio-http, ZIO JDBC, ZIO Streams, and documented every deviation in ADRs.

- **Uniform composition**

- Services assemble from ZLayers (typed DI); live state is a SubscriptionRef; shutdown is a Promise; errors ride a typed IO[GameError, A] end-to-end.

- **Pure domain, effects at the edges**

- The domain stays pure; effects live only at the boundaries (ADR-005).

ZLayer

SubscriptionRef

Promise

ZStream

IO[E,A]

 The payoff**No adapter glue.**

SSE feed, Kafka consumer, gRPC server and DB layer all speak the same ZIO/ZStream/ZLayer vocabulary — they wire together without impedance mismatches.



ZIO, in code

```
// typed dependency injection — services assemble from ZLayers
program.provideSome[Scope](
  Server.defaultWithPort(httpPort),
  Client.default,
  tracedClientLayer,
  TracingLayer.fromEnv("gateway")
)

// one authoritative state, subscribable — SSE with zero polling
SubscriptionRef.make(SessionState(snapshot)) // game-service

ref.changes.mapZIO: state => // gateway SSE feed
  GrpcMappers.toStateReply(gameId, state)
```

gRPC, and the wire-format war story

Measured during
the migration

zio-grpc internal RPC

State first crossed the wire as a FEN string — it lost UI info and forced the gateway to re-parse and re-derive FEN on every reply. (ADR-011)

Structured BoardStateDto

Now carried as boopickle bytes in the gRPC StateReply: full board, move log, status — no gateway re-derivation.

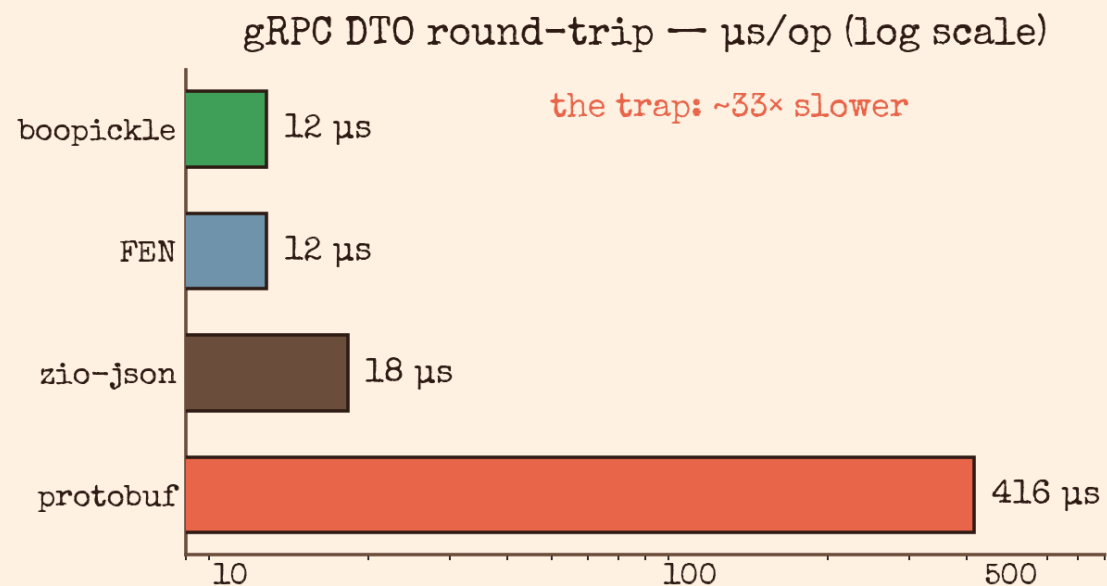
⚠ The protobuf trap

zio-schema-protobuf benched ~33× slower (~416 vs ~12 μs) and regressed p99 ~7× under load. We caught it and backed it out.

Codec tier (μs/op)

boopickle ~12 · FEN ~12 · json ~18 · protobuf ~416. boopickle ties FEN on speed — the win is structural, not raw codec speed.

The codec, measured



```
given Pickler[BoardStateDto] = generatePickler

def encodeBytes(dto: BoardStateDto): Array[Byte] =
  val buf = Pickle.intoBytes(dto)    // ~12 us round-trip
  val arr = new Array[Byte](buf.remaining)
  buf.get(arr); arr

def decodeBytes(bytes: Array[Byte]): BoardStateDto =
  Unpickle[BoardStateDto].fromBytes(ByteBuffer.wrap(bytes))
```

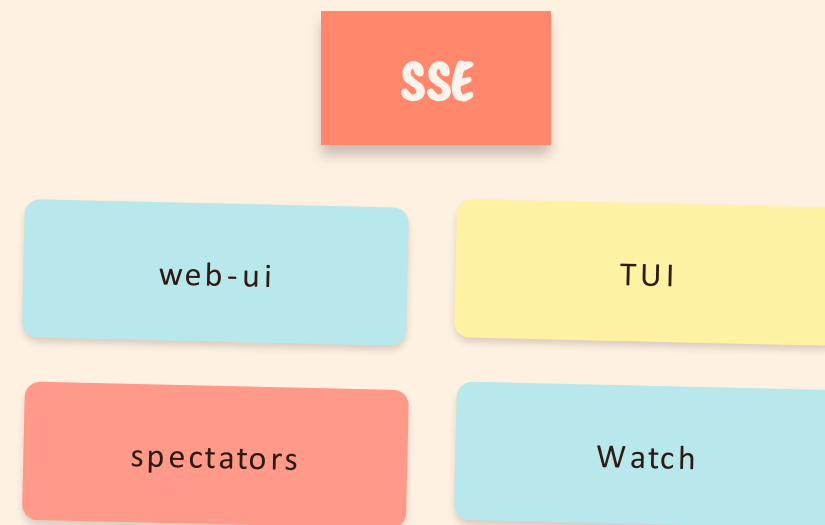
protobuf benched ~33x slower and was backed out; boopickle ties FEN (~12 μs) - the win is structural.



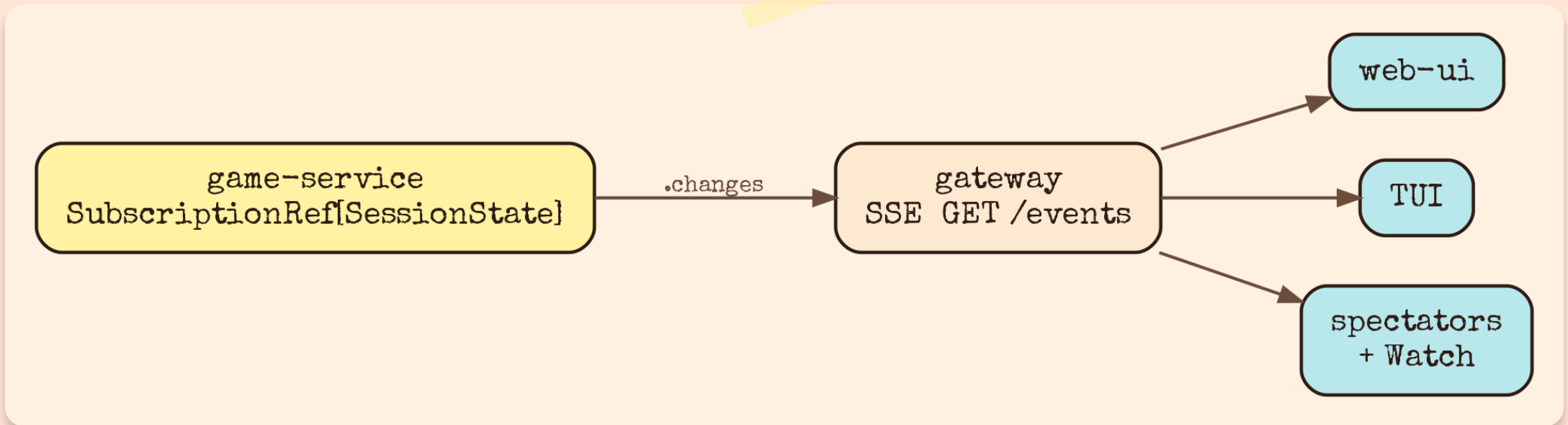
One state, two surfaces, zero polling

- **One authoritative state**
- game-service holds a `SubscriptionRef[SessionState]`; the gateway streams `.changes` over SSE at `/api/games/{id}/events`.
- **Move in the terminal, see it in the browser**
- Both web-ui and TUI subscribe — a move in either shows in the other instantly, no polling.
- **Same feed carries presence & Watch**
- Live spectator presence and policy-gated Watch ride the same per-game feed.
- **Coordinated shutdown**
- A shared Promise — quit from any surface ends them all.

one state → many subscribers



One state, many subscribers



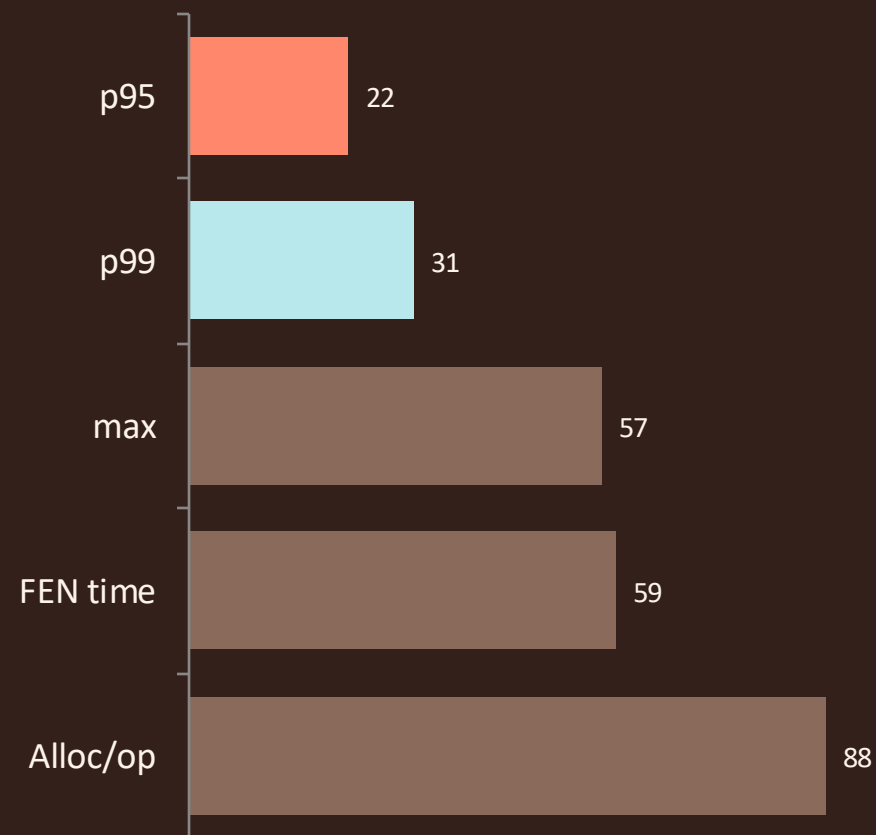
A single SubscriptionRef fans out over SSE to web-ui, TUI and spectators - zero polling, coordinated shutdown.



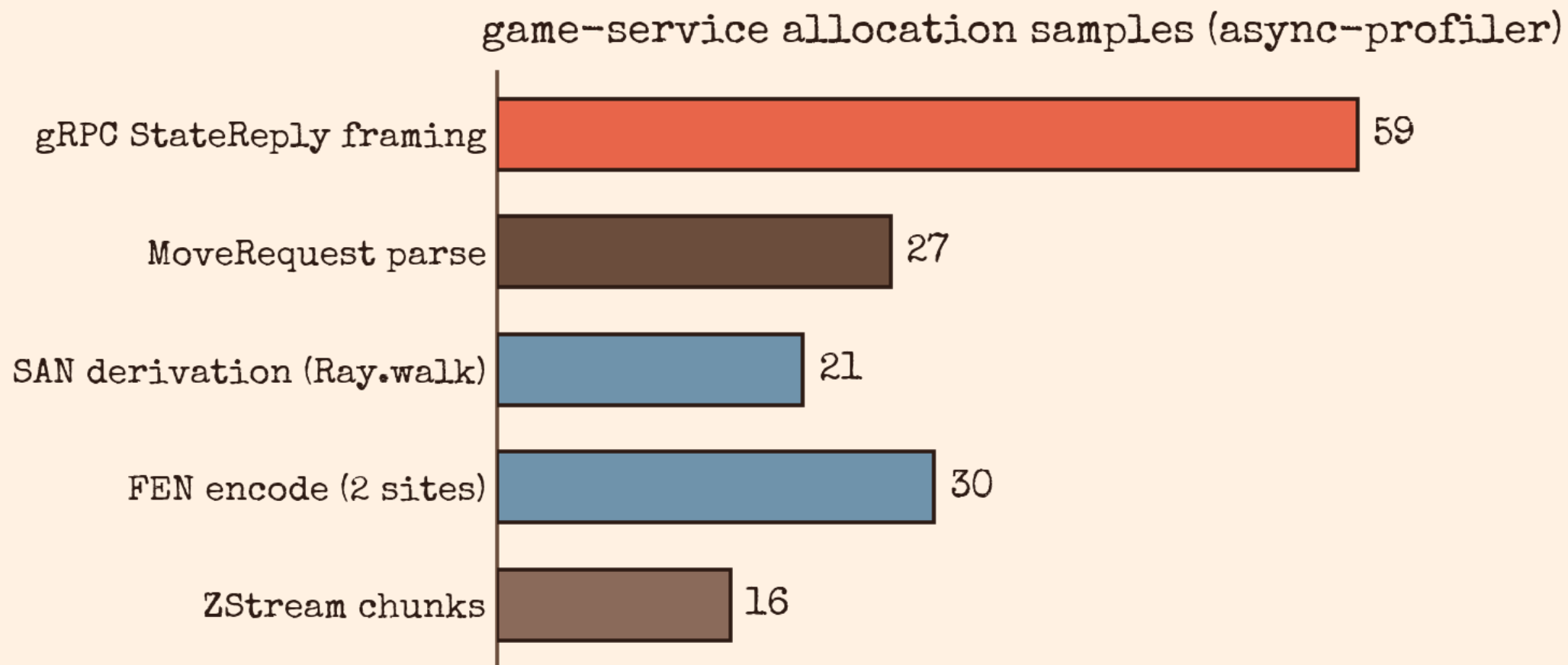
Measured, not guessed

- **–88% allocation on FEN serialize**
- 40,384 → 4,896 B/op (per-cell Range.map → pre-sized StringBuilder) — the biggest single number.
- **Tail latency down across the board**
- End-to-end p99 –31% (13→9 ms), p95 –22% (9→7 ms), max –57% (30→13 ms).
- **The method, not a hunch**
- async-profiler: GC ate ~30% of on-CPU samples → JMH isolated each hot path → fix → re-bench.
- **Optimisation[T] A/B framework**
- Every fix is a PICHESS_OPT_* toggle resolved to a ZLayer — and HikariCP pooling lost its A/B (72 vs 4 ms p95), so we shipped the simpler impl.

improvement % (higher = bigger win)



Where the bytes are born

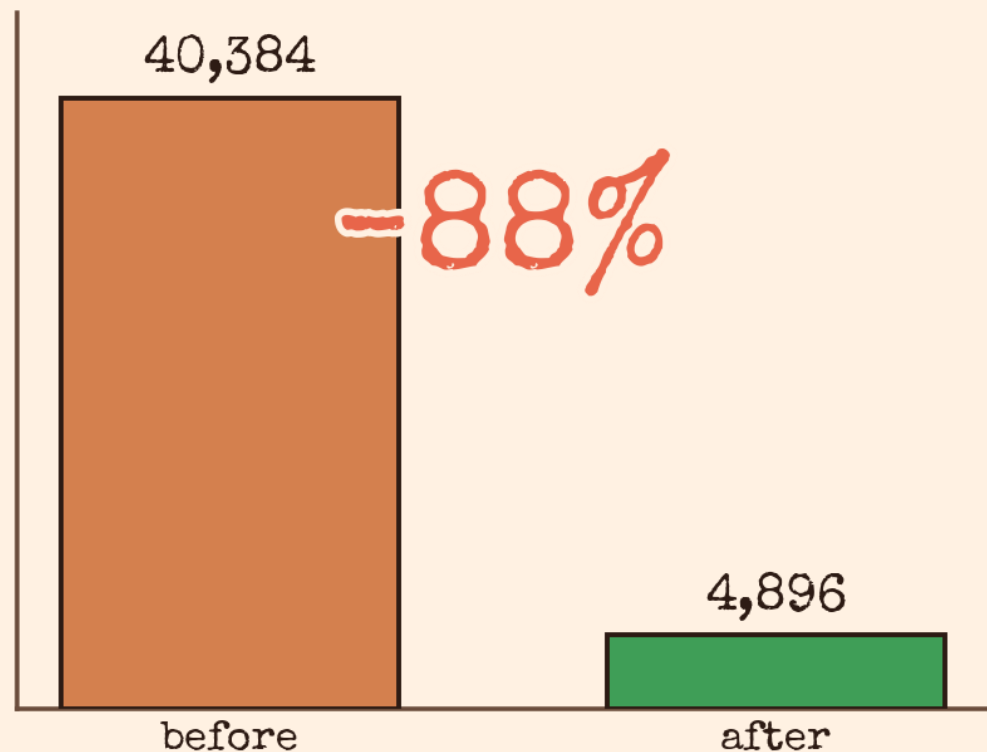


async-profiler: ~30% of on-CPU samples were GC. The response path - not persistence - was the allocator.

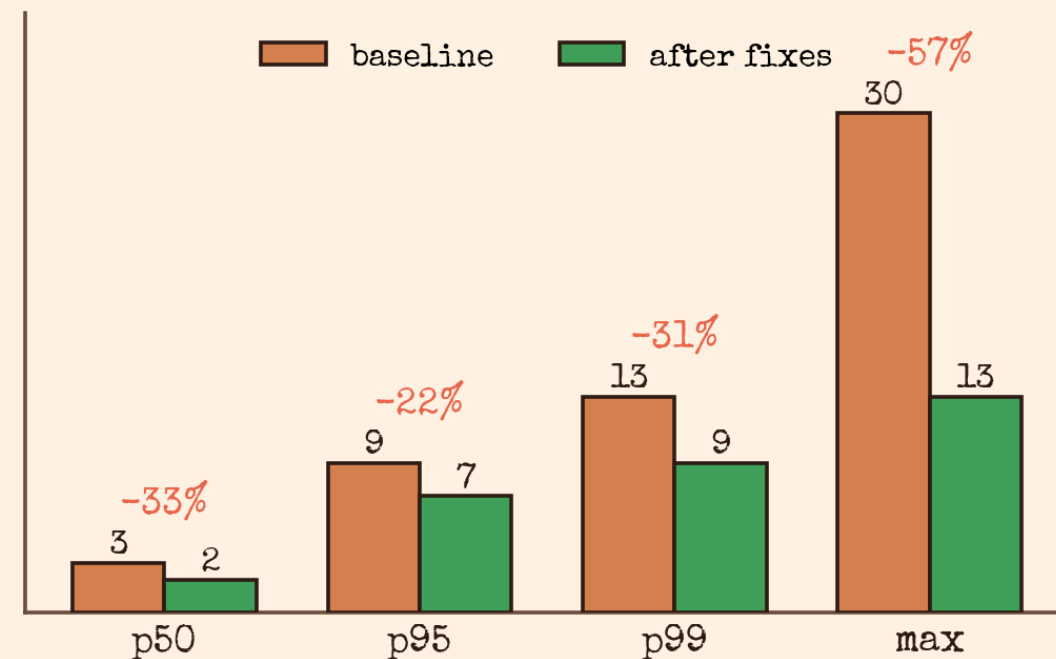


The wins, measured

FEN serialize — bytes allocated / op



End-to-end Stress latency (ms) · lower = better



OPTIMISATION

...in code

```
// -88% allocation: per-cell Range.map -> one pre-sized StringBuilder
def encodeBoard(board: Board): String =
  val sb = new java.lang.StringBuilder(72)
  var row = 8
  while row >= 1 do
    var col = 'a'; var empty = 0
    while col <= 'h' do
      board.get(Position(col, row)) match
        case None      => empty += 1
        case Some(piece) =>
          if empty > 0 then { sb.append(empty); empty = 0 }
          sb.append(pieceToChar(piece))
      col = (col + 1).toChar
    if empty > 0 then sb.append(empty)
    if row > 1 then sb.append("/"); row -= 1
  sb.toString
```

```
// fix #3 — 64 cached Positions built once at class-load (flyweight)
def apply(col: Char, row: Int): Position =
  cached((col - 'a') * 8 + (row - 1)) // no per-cell allocation

// every optimisation ships as an A/B pair, selected by env -> ZLayer
trait Optimisation[T]:
  def default: ZLayer[Any, Throwable, T]
  def baseline: ZLayer[Any, Throwable, T]

given Optimisation[PostgresDatabase] with
  val name      = "PG_POOL"
  val default   = PostgresDatabase.withSchemaLayerHikari
  val baseline  = PostgresDatabase.withSchemaLayer // Hikari lost the A/B
```

Five backends, one DAO seam

SQL

Postgres (Slick)

Relational primary behind GameRepository.

DOC

Mongo

Prod primary — sole combo at 100% success, sub-1.5s p99 under 970-RPS load.

KV

Redis

In-memory KV + cache decorator over any primary.

COL

Cassandra

Wide-column backend, same trait.

RAM

In-memory

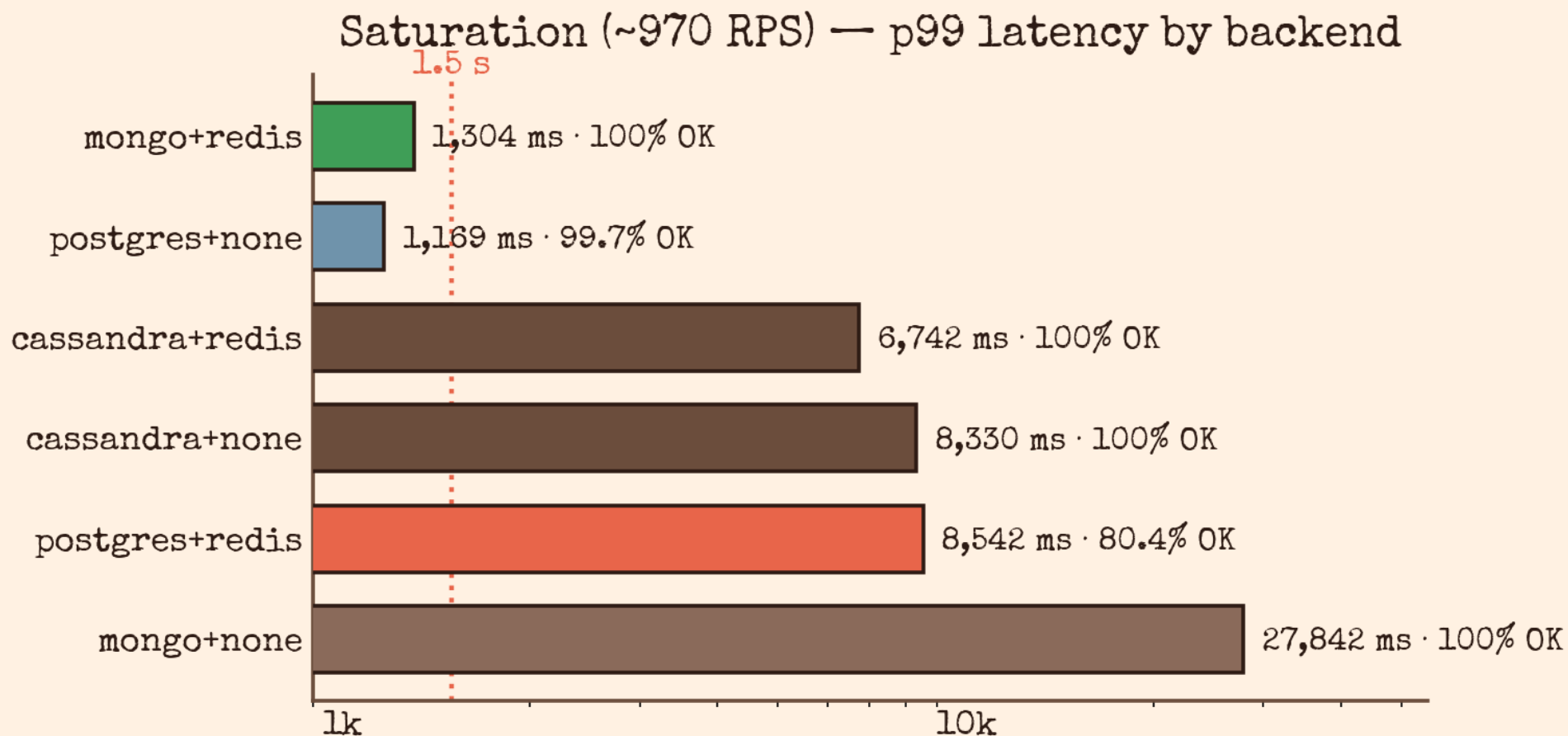
Zero-dependency backend for tests & dev.

TEST

One contract suite

Testcontainers proves all five are identical (ADR-016).

The default, chosen by experiment



At ~970 RPS, mongo+redis was the only durable combo at 100% with sub-1.5s p99; postgres+redis fell to 80.4%.



A real Lambda + CQRS pipeline

- **Speed layer**
- Spark Structured Streaming sessionizes each game (flatMapGroupsWithState) + event-time windowed counts with watermarks → GameSummary to chess.analytics.
- **Batch layer**
- Archived events → aggregates → authoritative Parquet views.
- **Two independent CQRS read-sides**
- Separate Kafka consumer groups: a Neo4j opening-move graph and a Prometheus-metrics service — each rebuildable in isolation (ADR-018).

zio-spark

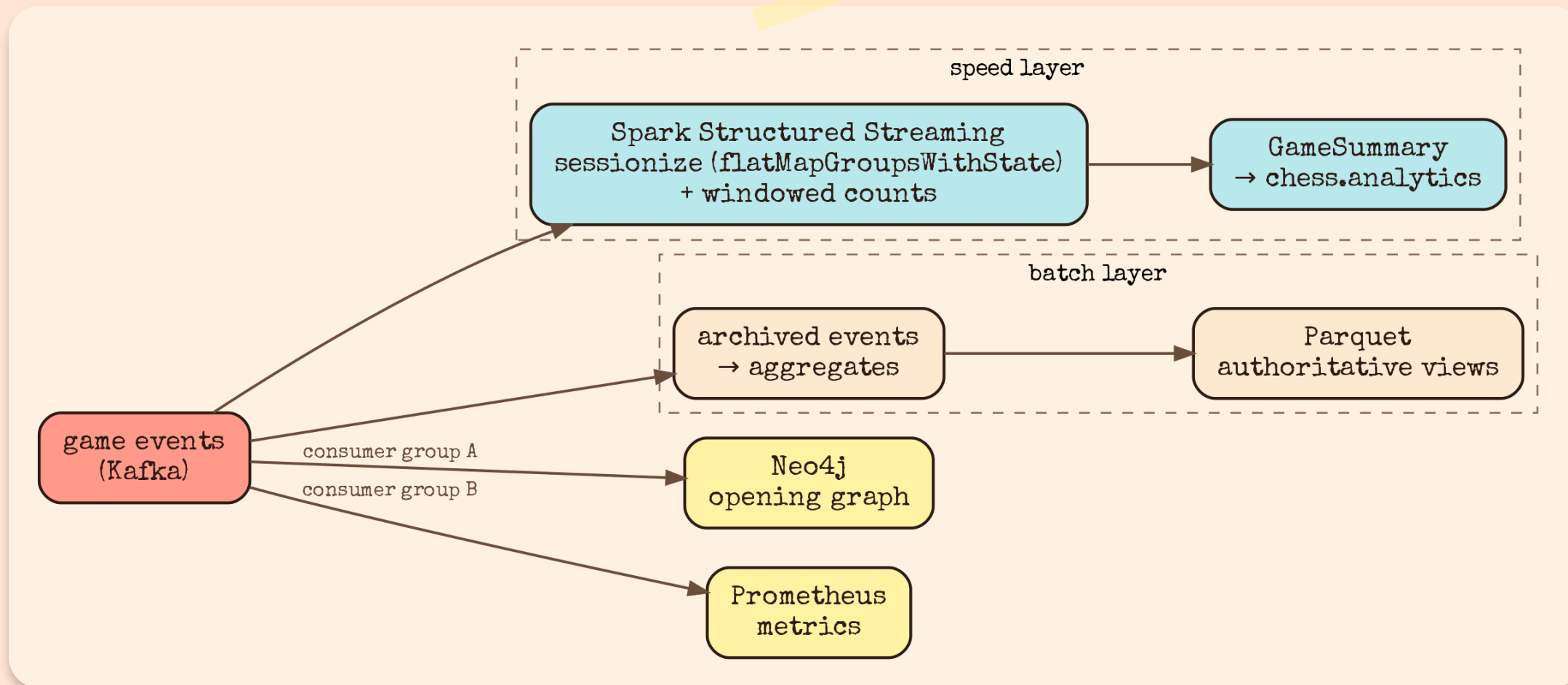
Effect-typed Spark, with a documented Scala-3.3 / Java-17 compatibility seam (ADR-022). The brief was just “read a file, then read Kafka.”

Spark

Neo4j

Parquet

The Lambda + CQRS pipeline



make help → 89 targets

```
$ make help
```

```
PERSISTENCE          stack-mongo stack-postgres stack-cassandra stack-redis stack-inmemory
DB CONSOLES          psql mongo redis-cli cqlsh cypher
PERF LAB             perf db-matrix profile-async-cpu k6 bench-codec bench-rules
OBSERVABILITY        dev-obs prod-obs grafana jaeger prometheus
BOT TRAINING         nnue-data hce-distill nnue-retrain policy-prior ab-sweep
PER-SERVICE          build-* dev-* shell-* (x7 services)
DEPLOY / DEV         up down stack-status tui tailwind-watch
```

Observability & ops, done properly



Distributed tracing

transparent in Kafka headers → one Jaeger trace spans gateway → game-service → Kafka → repo, across the async hop.



make help → 89 targets

The whole project drives from one self-documenting Makefile — the DAO seam made visible at the CLI.



One-command swaps

stack-postgres / mongo / cassandra / redis, plus a full perf lab: perf, db-matrix, profile-async, k6, bench-*.



Ansible → k3s, not compose up

Local-driven, idempotent deploy to an HTWG VM behind a 2FA VPN; host-k3s or unprivileged k3d, same deploy.yml.



Security-conscious IaC

Throwaway deploy key (asserts it isn't a PRIVATE KEY), no agent forwarding, pulls only public GHCR — no creds reach the box.



Multi-arch images ship only if the **100% coverage** gate passes

The part a Software Architecture course grades

22

A 22-ADR decision log

Every significant choice written up with context, alternatives & consequences — superseded ones flagged.

100%

Test coverage, enforced

The build fails below 100% on every JVM module — cross-module aggregate gate + Testcontainers contract tests.

12GB

Memory-fit engineering

Profiled & tuned to fit a 4-core / 12 GB VM — Kafka heap capped, analysis depth-capped & serialized.

0

Runtime external APIs

NNUE nets, opening books & fonts all trained / vendored / committed — fully reproducible offline.



Process as deliberate
as the product.

THE GUI

A real design language



light theme



dark theme

The real app, re-themed light ↔ dark purely by CSS tokens: a notebook metaphor, sticky-note controls, Special Elite move log — all sharing DTOs with the JVM gateway via Laminar.





Thank you

piChess — a chess platform that grew up, one architecture at a time.



Code: github.com/jonaskromer/pichess

Questions?

